LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Gerrard
NIM : 2428240067
Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Gerrard
NPM	2428240067
Kelas	SI5B
Pertemuan Ke-	3
Topik / Judul Praktikum	**Tugas 1 RESTful API**
Tanggal Pelaksanaan	2026-09-29
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Membuat project Express.js sebagai fondasi awal pembuatan aplikasi *backend*
2. Membuat routing RESTful untuk memetakan URL dan metode HTTP ke fungsi penangan (*handler*)
3. Menerima request dari client melalui parameter rute (req.params), *query string* (req.query), dan *body request* (req.body)
4. Mengirimkan response murni dalam format JSON beserta penggunaan kode status HTTP yang sesuai
5. Mengimplementasikan RESTful API secara utuh dengan menggabungkan konsep *routing*, penanganan *request*, dan *response* JSON murni
6. Mengembangkan RESTful API murni untuk pengelolaan *resource* stok darah PMI (/blood-stocks) yang mencakup operasi CRUD lengkap, validasi data, *catch-all middleware* 404, serta *deployment* ke Vercel

II. Dasar Teori

A. Environment & Runtime (Node.js, NVM, dan NPM)

Node.js merupakan *runtime environment* JavaScript berbasis mesin V8 milik Google Chrome yang memungkinkan kode JavaScript dijalankan di luar peramban (*sisi server*). Node.js menjadi fondasi utama untuk menjalankan Express.js dan ekosistem paket npm. NVM (Node Version Manager) merupakan alat untuk menginstal dan mengelola beberapa versi Node.js secara fleksibel pada satu sistem, memastikan seluruh pengembang dalam tim bekerja pada versi Node.js yang konsisten. NPM (Node Package Manager) merupakan manajer paket bawaan Node.js yang digunakan untuk menginstal, memperbarui, dan mengelola

dependensi/paket pustaka yang tercatat dalam berkas package.json dan package-lock.json

B. Express.js Framework

Express.js merupakan *framework* web minimalis dan fleksibel untuk Node.js yang mempermudah pembangunan *server* web dan RESTful API tanpa menulis banyak kode *boilerplate*. Fitur utamanya meliputi pemetaan rute (*routing*) sederhana untuk menghubungkan URL dan metode HTTP ke fungsi penangan (*handler*), dukungan komponen perantara (*middleware*), serta fleksibilitas struktur proyek

C. Arsitektur RESTful API

REST (*Representational State Transfer*) adalah gaya arsitektur untuk merancang *web service* berbasis *resource* (sumber daya) yang diakses melalui protokol HTTP. Prinsip utamanya meliputi:

a. Resource-Based: Setiap entitas data (seperti /blood-stocks) direpresentasikan melalui URL unik berupa kata benda jamak
b. Stateless: Setiap permintaan (*request*) bersifat independen, sehingga *server* tidak menyimpan status sesi (*session state*) klien
c. Pemetaan HTTP Method ke Operasi CRUD:
    - GET: Membaca seluruh koleksi data atau data spesifik berdasarkan ID tanpa mengubah data
    - POST: Membuat/menambahkan data baru ke *server*
    - PUT: Memperbarui/mengganti seluruh *field* data yang ada secara penuh berdasarkan ID
    - DELETE: Menghapus data spesifik berdasarkan ID

D. Anatomi Request, Response, dan Middleware Express.js

- Objek Request (req) digunakan untuk membaca data masuk dari klien melalui req.params (parameter rute dinamis), req.query (*query string*), dan req.body (data pada *body request*).
- Middleware express.json() Komponen perantara wajib yang mendaftarkan penguraian (*parsing*) JSON sebelum rute agar aplikasi dapat membaca req.body.
- Objek Response (res) dan Kode Status HTTP digunakan untuk mengirimkan respons kembali ke klien beserta status HTTP yang sesuai. 2xx (Berhasil): 200 OK (berhasil mengambil/mengubah data), 201 Created (berhasil membuat data baru), 204 No Content (berhasil menghapus data). 4xx / 5xx (Gagal/Error): 400 Bad Request, 404 Not Found (data/rute tidak ditemukan), atau 500 Internal Server Error

E. Deployment Server (Vercel)

Platform Vercel menyediakan lingkungan *cloud serverless* untuk mengeksekusi aplikasi Express.js. Aplikasi diekspor menggunakan module.exports = app dan dikonfigurasi melalui berkas vercel.json agar *serverless function* dapat memetakan rute secara otomatis

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	*Runtime environment* JavaScript untuk menjalankan *server* aplikasi sisi *backend*
2	NPM (Node Package Manager)	Manajer paket bawaan Node.js untuk mengelola instalasi dependensi proyek
3	Express.js	Version ^5.2.1 (*framework web* utama untuk membangun RESTful API)
4	Nodemon	Version ^3.1.14 (*devDependency* untuk memuat ulang *server* secara otomatis saat terjadi perubahan kode)
5	Visual Studio Code	*Code editor* utama yang digunakan untuk penulisan dan pengelolaan kode proyek
6	Git & GitHub	*Version control system* dan repositori publik (tugas1-restful-2428240032) untuk pengelolaan riwayat *commit*
7	Vercel	Platform *cloud deployment* berbasis *serverless* untuk meng-host RESTful API secara *online*
8	Postman	Alat penguji API (*API testing tool*) untuk menjalankan 10 skenario pengujian *endpoint*

IV. Langkah Kerja

1. Inisialisasi Proyek dan Instalasi Dependensi

   Langkah awal diawali dengan membuat folder proyek baru bernama tugas1-restful-2428240067 melalui terminal dan masuk ke dalam direktori tersebut. Proyek Node.js kemudian diinisialisasi menggunakan perintah npm init -y sehingga menghasilkan berkas konfigurasi package.json. Selanjutnya, *framework* Express.js dipasang sebagai dependensi utama menggunakan perintah npm install express, sedangkan Nodemon dipasang sebagai *devDependency* menggunakan perintah npm install --save-dev nodemon untuk membantu proses pengembangan dengan memuat ulang *server* secara otomatis ketika terjadi perubahan kode.

2. Konfigurasi package.json , .gitignore , dan vercel.json

   Berkas package.json dikonfigurasi untuk mengatur informasi proyek, dependensi, serta *script* yang digunakan untuk menjalankan aplikasi. Script "start": "node app.js" digunakan untuk menjalankan *server* secara normal, sedangkan script pengembangan menggunakan Nodemon. Selanjutnya, berkas .gitignore dibuat untuk mengecualikan folder node_modules/ dan berkas .env agar tidak ikut diunggah ke repository GitHub. Berkas vercel.json juga disiapkan pada direktori utama proyek dengan konfigurasi @vercel/node untuk mengatur proses *deployment* aplikasi Express.js ke platform Vercel. Ketentuan tugas memang mensyaratkan adanya .gitignore, vercel.json, serta konfigurasi aplikasi agar dapat berjalan secara *serverless* di Vercel.

3. Inisialisasi Server, Middleware, dan Data Awal (app.js)

   Berkas app.js digunakan sebagai *entry point* utama aplikasi. Pada bagian awal, pustaka Express.js diimpor kemudian dibuat *instance* aplikasi menggunakan express(). Middleware app.use(express.json()) digunakan agar *server* dapat membaca dan mengolah data masukan dalam format JSON melalui req.body. Selanjutnya dibuat data sementara berbasis memori (*in-memory array*) bernama shipments yang digunakan untuk menyimpan data paket kiriman. Setiap data memiliki atribut id, nomorResi, pengirim, penerima, beratKg, dan statusPengiriman. Status pengiriman yang digunakan adalah "diproses", "dikirim", dan "diterima" sesuai dengan ketentuan Topik 12 – Ekspedisi: Paket Kiriman.

4. Implementasi Endpoint Informasi API (GET /)

   Endpoint GET / dibuat sebagai rute utama untuk memberikan informasi mengenai API yang telah dibuat. Endpoint ini digunakan untuk memastikan bahwa *server* dapat menerima request dengan baik sekaligus memberikan informasi dasar mengenai aplikasi dan endpoint yang tersedia. Response yang diberikan menggunakan format JSON sehingga sesuai dengan ketentuan RESTful API yang tidak menggunakan HTML atau *template engine*.

5. Pembuatan Rute CRUD dan Filter Query String (/blood-stocks)
   Membangun enam rute utama sesuai standar konvensi RESTful API:
   Selanjutnya dibuat enam rute utama untuk mengelola resource paket kiriman sesuai dengan konvensi RESTful API. Rute GET /shipments digunakan untuk mengambil seluruh data paket dalam bentuk *array* JSON. Rute GET /shipments/:id digunakan untuk mengambil satu data paket berdasarkan id yang dikirim melalui req.params.id; apabila data tidak ditemukan, server memberikan response 404 Not Found. Rute GET /shipments?statusPengiriman=dikirim menggunakan

req.query.statusPengiriman untuk melakukan penyaringan data berdasarkan status pengiriman. Rute POST /shipments digunakan untuk menambahkan paket baru berdasarkan data yang dikirim melalui req.body dan memberikan ID secara otomatis, kemudian mengembalikan response 201 Created. Rute PUT /shipments/:id digunakan untuk memperbarui seluruh data paket berdasarkan ID dan memberikan response 200 OK. Terakhir, rute DELETE /shipments/:id digunakan untuk menghapus data paket berdasarkan ID dan memberikan response 200 OK dengan data: null. Keenam operasi tersebut sesuai dengan struktur endpoint yang diwajibkan pada tugas.

6. Penerapan Fungsi Validasi Data Masukan

   Fungsi validasi data diterapkan pada proses penambahan dan perubahan data paket melalui endpoint POST /shipments dan PUT /shipments/:id. Validasi dilakukan untuk memastikan seluruh *field* wajib, yaitu nomorResi, pengirim, penerima, beratKg, dan statusPengiriman, telah diberikan oleh pengguna. Selain itu, tipe data setiap *field* diperiksa agar sesuai dengan ketentuan, yaitu data berupa teks untuk nomor resi, pengirim, dan penerima, angka untuk beratKg, serta nilai statusPengiriman hanya berupa "diproses", "dikirim", atau "diterima". Apabila terdapat data yang tidak lengkap atau tidak sesuai, server memberikan response 400 Bad Request beserta pesan kesalahan dalam format JSON.

7. Penanganan Middleware Catch-All 404

   Pada bagian akhir app.js ditambahkan *middleware catch-all* untuk menangani request menuju endpoint yang tidak terdaftar. Middleware ini berfungsi agar setiap request yang menggunakan URL yang tidak tersedia tidak menghasilkan halaman HTML bawaan Express, tetapi mengembalikan response JSON dengan struktur { status, message, data }. Response yang diberikan menggunakan status 404 Not Found dengan pesan "Endpoint tidak ditemukan". Mekanisme ini juga sesuai dengan ketentuan tugas yang mengharuskan endpoint yang tidak terdaftar ditangani menggunakan response JSON.

8. Pengaturan Ekspor Serverless dan Listener Port Lokal

   Aplikasi dikonfigurasi agar dapat berjalan pada lingkungan lokal maupun Vercel. Pada lingkungan lokal, server dijalankan menggunakan app.listen() dengan port yang berasal dari process.env.PORT atau menggunakan port 3000 sebagai nilai bawaan. Agar dapat digunakan pada lingkungan *serverless*, app.listen() hanya dijalankan ketika aplikasi tidak berada pada mode production. Selanjutnya, *instance* Express diekspor menggunakan module.exports = app; sehingga Vercel dapat menggunakan aplikasi tersebut sebagai *serverless function*. Konfigurasi ini merupakan bagian yang dipersyaratkan dalam proses deployment RESTful API ke Vercel.

9. Version Control Git dan Deployment Vercel

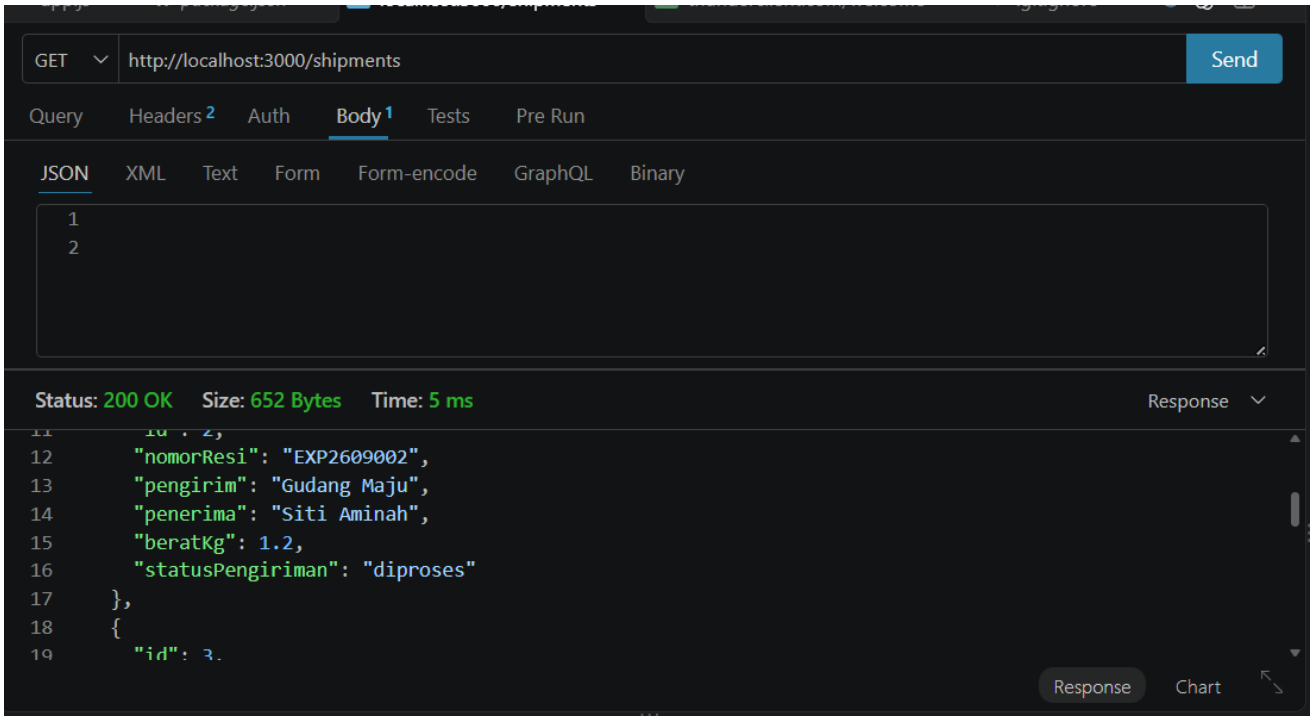
   Proyek diinisialisasi menggunakan Git dengan perintah git init, kemudian setiap perubahan pada pengembangan RESTful API disimpan melalui proses *commit* secara bertahap dengan minimal 5 kali *commit*. Setelah seluruh kode selesai, proyek di-*push* ke repository GitHub publik dengan nama tugas1-restful-2428240067. Selanjutnya, repository GitHub dihubungkan dan diimpor ke platform Vercel untuk melakukan proses *deployment*. Setelah proses *deployment* berhasil, RESTful API Paket Kiriman dapat diakses secara online melalui URL Vercel dan endpoint API dapat diuji kembali untuk memastikan seluruh fungsi berjalan dengan baik.

V. Hasil dan Pembahasan

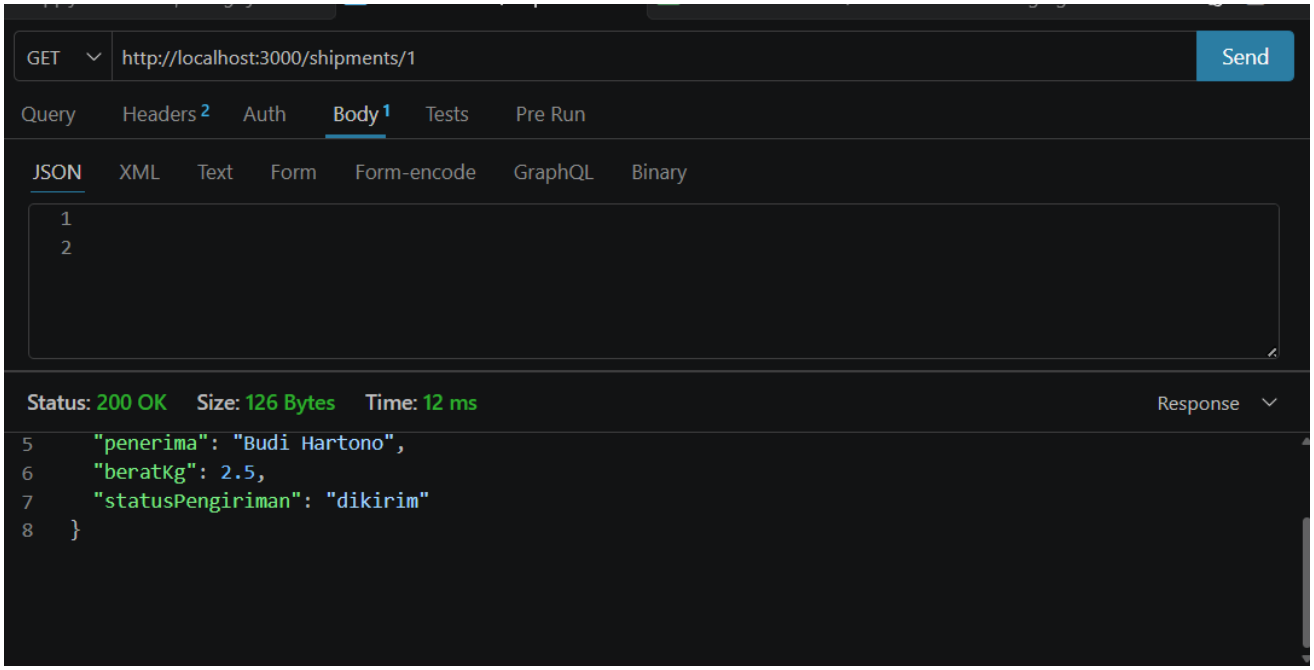
1. Tabel Hasil Pengujian API (Vercel & Postman Collection)

Berikut adalah rangkuman hasil pengujian 10 skenario *endpoint* API yang dijalankan pada URL *deployment* Vercel (https://tugas1-restful-2428240067.vercel.app) menggunakan koleksi Postman 2428240067_Postman.json:

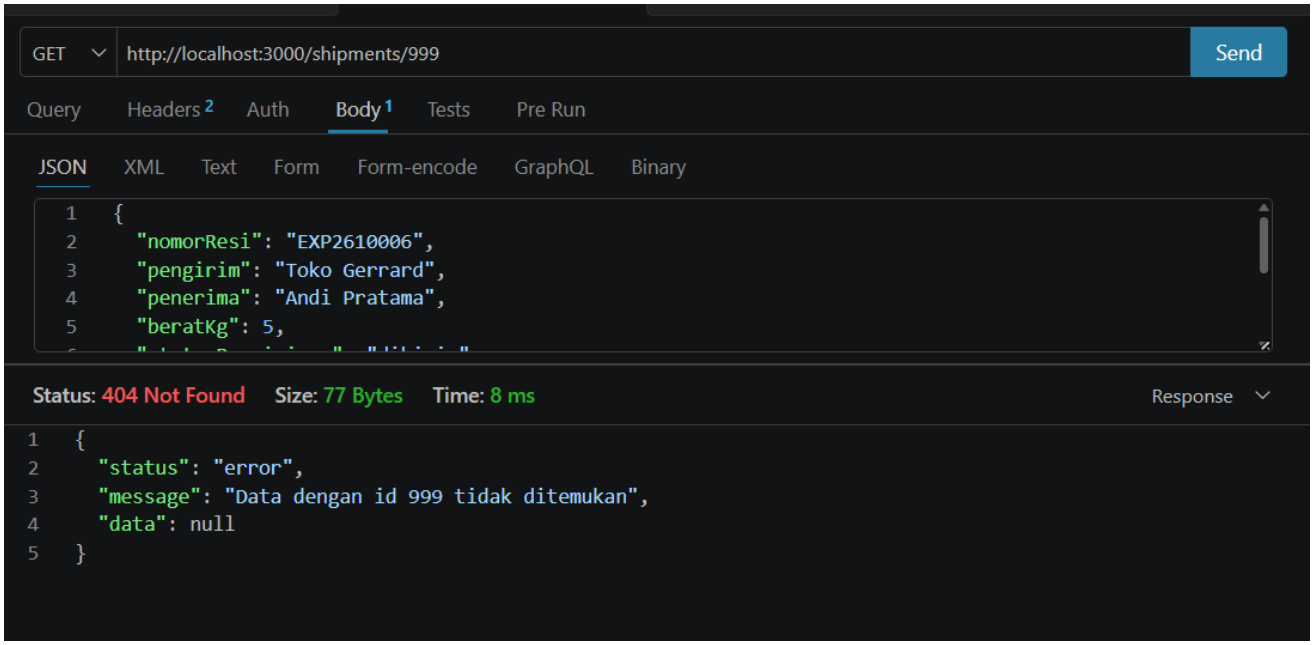
No.	Method	Endpoint / URL	Data Request	Status Diharapkan	Status Hasil
1.	GET	/shipments	(Tanpa Body)	200 OK	200 OK
2.	GET	/shipments/1	(Tanpa Body)	200 OK	200 OK
3.	GET	/shipments/999	(Tanpa Body)	404 Not Found	404 Not Found
4.	GET	/shipments?statusPengiriman=dikirim	(Tanpa Body)	200 OK	OK
5.	POST	/shipments	Body JSON Lengkap	201 Created	201 Created
6.	POST	/shipments	Field Wajib Kosong	400 Bad Request	400 Bad Request
7.	PUT	/shipments/1	Body JSON Lengkap	200 OK	200 OK
8.	PUT	/shipments/999	Body JSON Lengkap	404 Not Found	404 Not Found
9.	DELETE	/shipments/1	(Tanpa Body)	200 OK	200 OK
10.	DELETE	/shipments/999	(Tanpa Body)	404 Not Found	404 Not Found



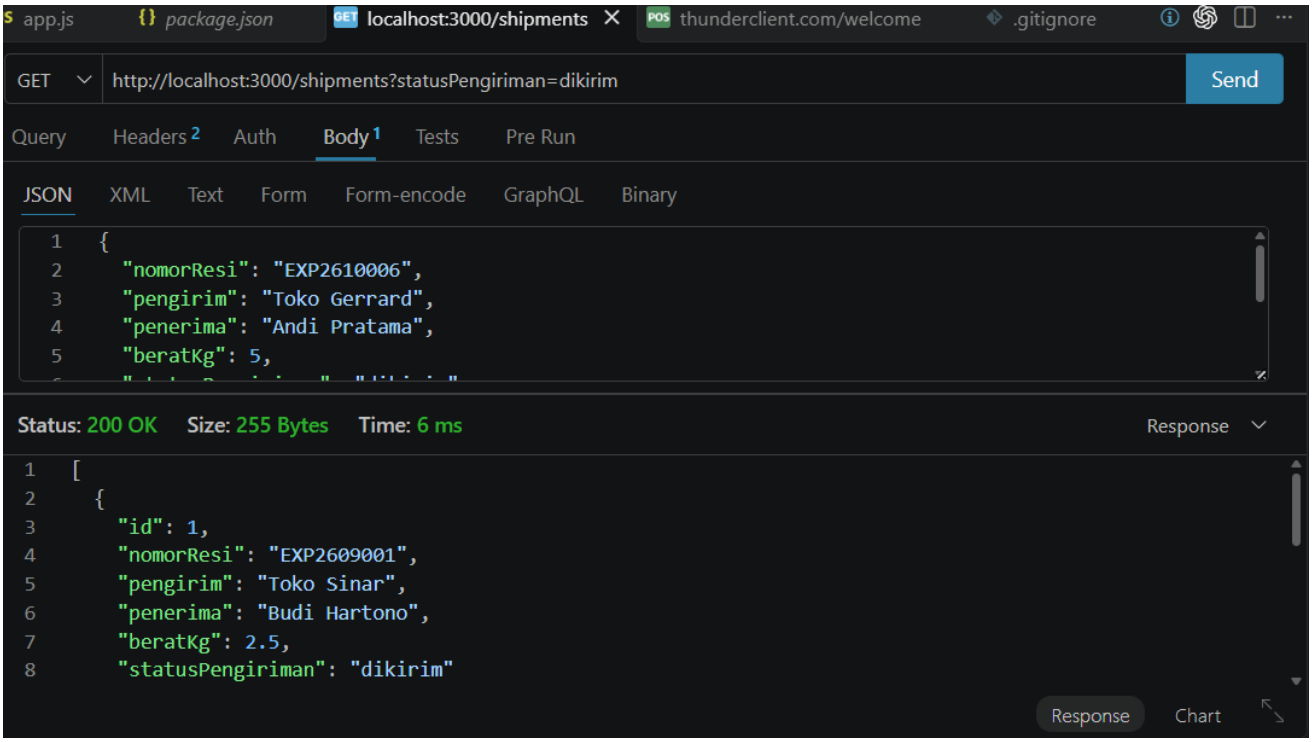
Gambar 5.1 *Hasil Pengujian GET /shipments (Mengambil Seluruh Data Paket Kiriman)*



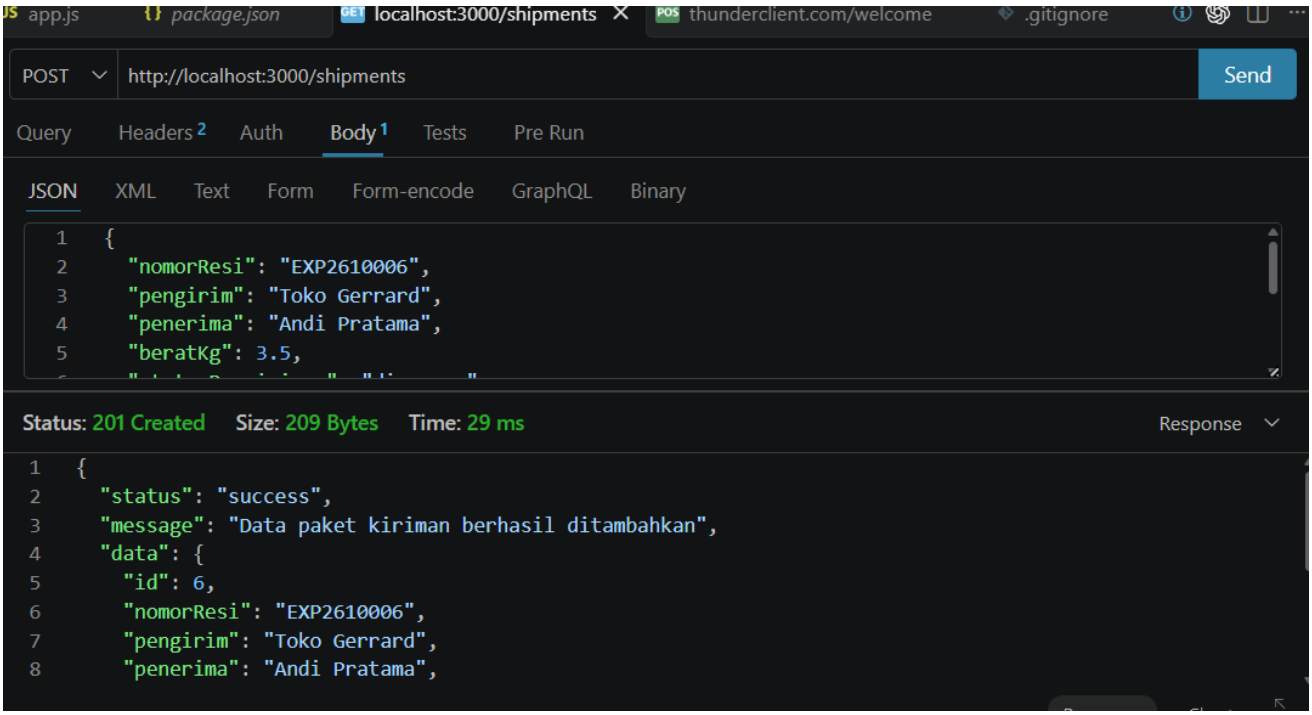
Gambar 5.2 *Hasil Pengujian GET /shipments/1 (Mengambil Detail Paket Kiriman Berdasarkan ID)*



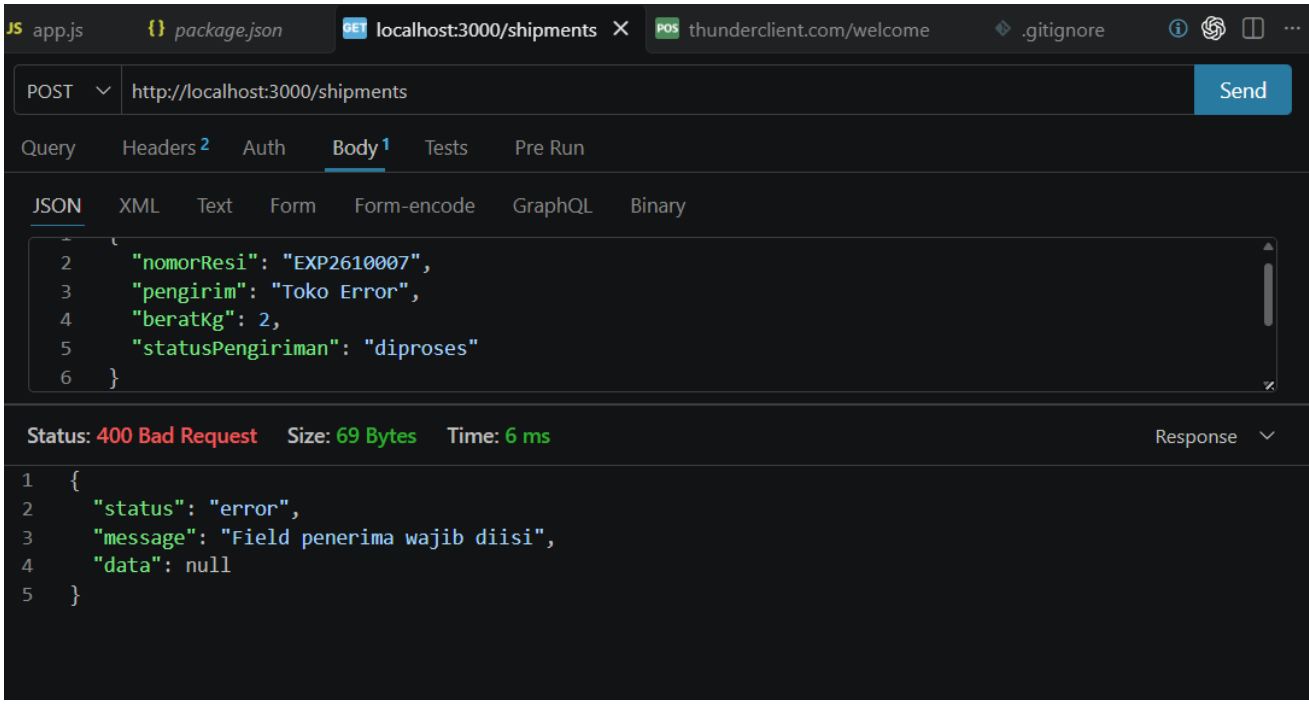
Gambar 5.3 *Hasil Pengujian GET /shipments/999 (Pengujian ID Paket Kiriman yang Tidak Ditemukan)*



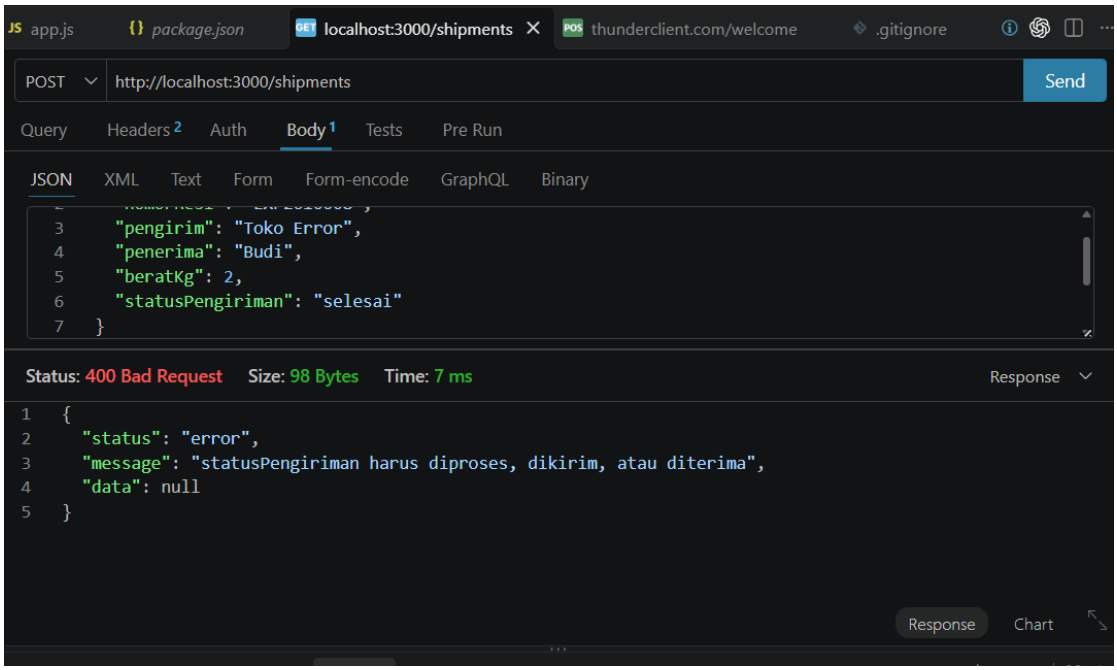
Gambar 5.4 *Hasil Pengujian GET /shipments?statusPengiriman=dikirim (Filter Paket Berdasarkan Status Pengiriman)*



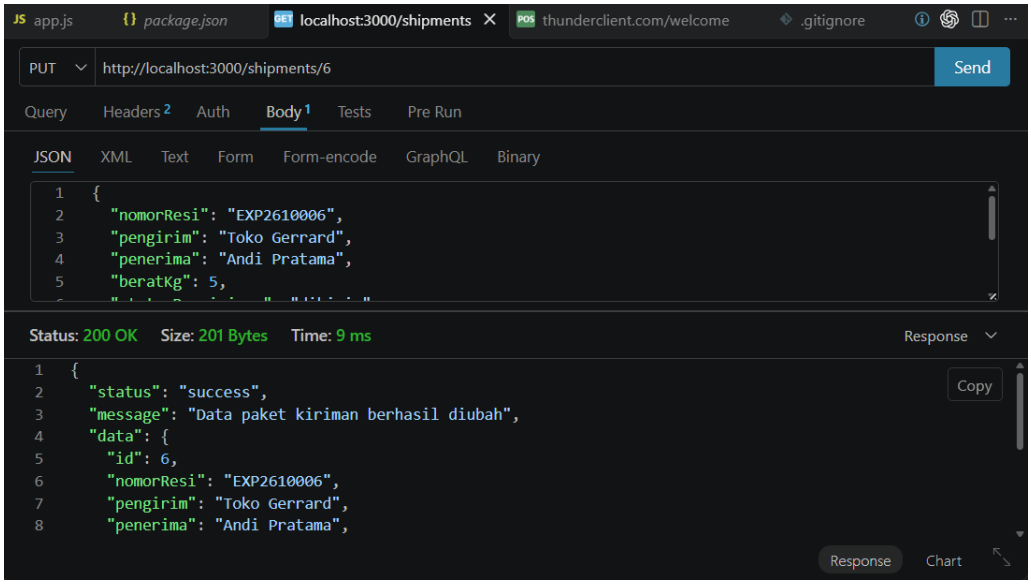
Gambar 5.5 *Hasil Pengujian POST /shipments (Menambahkan Data Paket Kiriman Baru)*



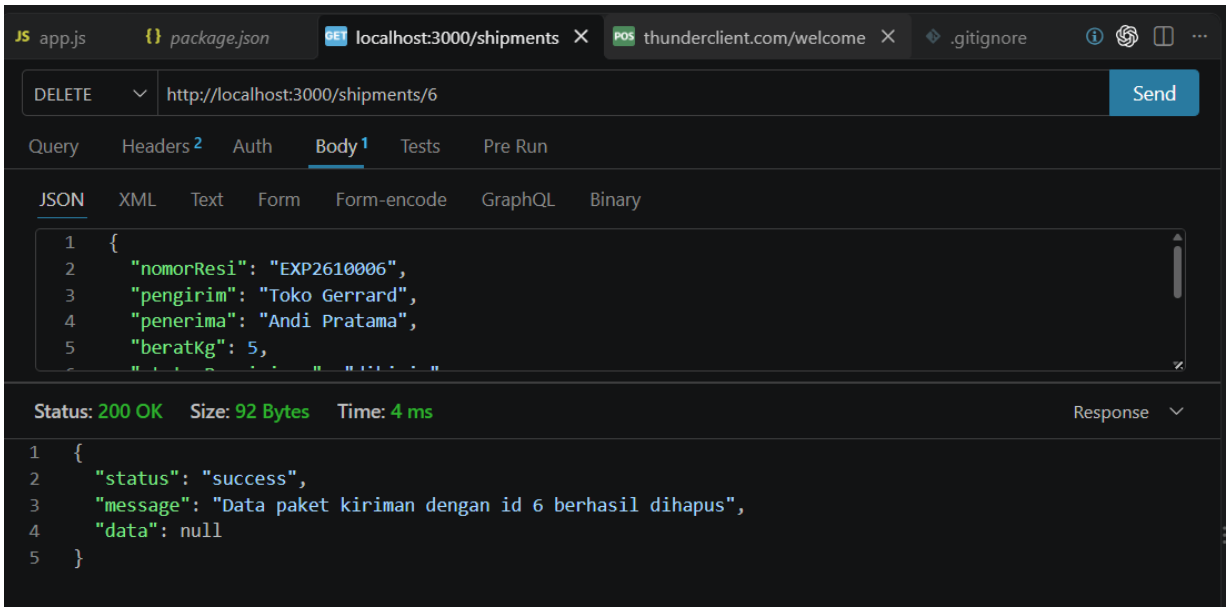
Gambar 5.6 *Hasil Pengujian POST /shipments (Validasi Data Paket Kiriman dengan Field Wajib Kosong)*



Gambar 5.7 *Hasil Pengujian POST /shipments (Validasi Status Pengiriman yang Tidak Valid)*



Gambar 5.8 *Hasil Pengujian PUT /shipments/1 (Memperbarui Data Paket Kiriman)*



Gambar 5.9 *Hasil Pengujian DELETE /shipments/1 (Menghapus Data Paket Kiriman)*

2. Pembahasan Analisis Implementasi API

a. Pengujian Endpoint Read (GET All, GET By ID, dan Query Filter)

Skenario GET `/shipments` digunakan untuk mengambil seluruh koleksi data paket kiriman. Sesuai konvensi tugas, *endpoint* ini mengembalikan data murni dalam bentuk *array* JSON yang berisi seluruh data paket kiriman tanpa dibungkus objek `status` maupun `message`.

Skenario GET `/shipments/:id` memanfaatkan parameter rute dinamis `req.params.id`. Ketika ID 1 dipanggil, *server* mencari data paket kiriman berdasarkan ID dan mengembalikan satu objek JSON dengan status **200 OK**. Jika dipanggil dengan ID 999 yang tidak tersimpan, *server* mengembalikan kode status **404 Not Found** dengan respons JSON yang berisi informasi bahwa data paket kiriman dengan ID tersebut tidak ditemukan.

Skenario **GET** `/shipments?statusPengiriman=dikirim` menggunakan *query string* `req.query.statusPengiriman`. *Server* menyaring *array* data paket kiriman menggunakan `.filter()` dan mengembalikan data yang memiliki status pengiriman **dikirim**. Apabila tidak terdapat paket dengan status yang sesuai, *server* tetap mengembalikan *array* kosong `[]` dengan status **200 OK**.

b. Pengujian Endpoint Create (POST) dan Validasi Input Data

Skenario **POST `/shipments`** digunakan untuk menambahkan data paket kiriman baru. *Body request* JSON berisi `nomorResi`, `pengirim`, `penerima`, `beratKg`, dan `statusPengiriman`. *Server* mengurai data menggunakan *middleware* `express.json()`, memberikan ID secara otomatis, kemudian memasukkan data baru ke dalam *array*. Respons dikembalikan dengan struktur standar `{ "status": "success", "message": "...", "data": { ... } }` dan status **201 Created**.

Pada skenario validasi **400 Bad Request**, apabila salah satu *field* wajib tidak dikirim atau data yang diberikan tidak sesuai dengan ketentuan, fungsi validasi akan menolak permintaan. Contohnya, apabila `penerima` tidak diisi atau `beratKg` tidak menggunakan tipe data angka, maka *server*

memberikan respons **400 Bad Request** beserta pesan kesalahan yang menjelaskan bagian data yang tidak valid.

c. Pengujian Endpoint Update (PUT) dan Penggantian Data Penuh

Sesuai ketentuan RESTful API, metode **PUT** digunakan untuk melakukan penggantian seluruh *field* data (*full replacement*). Saat mengirimkan permintaan ke **`/shipments/1`** dengan seluruh *field* baru, *server* terlebih dahulu mencari data berdasarkan ID. Jika ID ditemukan dan seluruh *field* valid, data lama diganti dengan data baru dan *server* memberikan respons **200 OK** beserta data hasil pembaruan.

Apabila ID paket kiriman tidak ditemukan, seperti pada **`/shipments/999`**, *server* tidak melakukan proses pembaruan dan mengembalikan respons **404 Not Found**.

d. Pengujian Endpoint Delete (DELETE) dan Response JSON Murni

Permintaan **DELETE `/shipments/1`** digunakan untuk menghapus data paket kiriman berdasarkan ID. *Server* mencari data berdasarkan ID kemudian menghapus data tersebut dari *array*. Berbeda dengan respons **204 No Content**, pada Tugas 1 ini operasi DELETE yang berhasil mengembalikan kode status **200 OK** beserta respons JSON yang berisi pesan konfirmasi dan `data: null`.

Jika ID paket kiriman yang ingin dihapus tidak tersedia, seperti **`/shipments/999`**, *server* tidak melakukan proses penghapusan dan mengembalikan respons error dengan status **404 Not Found**.

e. Analisis Middleware Catch-All 404 dan Pengaruh Serverless Vercel

Middleware Catch-All 404 digunakan untuk mencegah Express mengirimkan balasan HTML bawaan ketika pengguna mengakses URL yang tidak tersedia, misalnya **GET `/endpoint-salah`**. *Middleware* ini memastikan respons dari aplikasi tetap konsisten dalam format JSON sehingga sesuai dengan karakteristik RESTful API yang dikembangkan.

Pada deployment menggunakan **Vercel** dengan penyimpanan data berupa *in-memory array* tanpa basis data eksternal, data paket kiriman yang ditambahkan melalui POST tidak bersifat permanen. Apabila *serverless function* mengalami *idle* atau dibuat ulang oleh platform, data dapat kembali ke kondisi data awal yang terdapat pada aplikasi. Kondisi tersebut merupakan karakteristik penggunaan penyimpanan *in-memory* pada lingkungan *serverless* dan masih sesuai dengan batasan tugas yang tidak menggunakan database eksternal.

VI. Kendala dan Solusi

No.	Kendala Teknis	Solusi dan Penanganan
1	Pembacaan req.body bernilai undefined saat pengujian metode POST dan PUT.	*Mendaftarkan middleware bawaan app.use(express.json()) pada baris awal berkas app.js sebelum pendefinisian rute agar Express dapat mengurai data JSON masukan.*
2	Data paket kiriman baru hasil **POST/PUT** dapat kembali ke data awal setelah beberapa saat ketika API dijalankan pada Vercel	Kondisi tersebut merupakan karakteristik lingkungan *cloud serverless* Vercel karena data disimpan menggunakan *in-memory array* tanpa basis data eksternal. Kondisi ini masih sesuai dengan batasan penugasan yang tidak menggunakan database.
3	Permintaan URL yang salah awalnya mengembalikan respons teks/HTML bawaan dari Express.js.	Menambahkan *middleware catch-all 404* pada bagian akhir berkas app.js untuk memastikan setiap rute yang tidak dikenal memberikan respons JSON, yaitu { "status": "error", "message": "Endpoint tidak ditemukan", "data": null }.
4	Terjadi kegagalan validasi terhadap data paket kiriman yang dimasukkan, seperti field wajib yang kosong atau nilai statusPengiriman yang tidak sesuai.	Membuat fungsi validasi data paket kiriman untuk memastikan seluruh *field* wajib (nomorResi, pengirim, penerima, beratKg, dan statusPengiriman) telah diisi, beratKg bertipe *number*, serta statusPengiriman hanya menggunakan nilai **diproses**, **dikirim**, atau **diterima**.

VII. Kesimpulan

Berdasarkan pelaksanaan praktikum dan penyelesaian Tugas 1 **RESTful API Ekspedisi – Paket Kiriman**, dapat disimpulkan bahwa:

1. **Implementasi RESTful API Murni:** Aplikasi *backend* untuk pengelolaan data paket kiriman pada resource **/shipments** berhasil dibangun menggunakan **Node.js dan Express.js** tanpa menggunakan basis data eksternal maupun *template engine* HTML.

2. **Kepatuhan Konvensi dan Operasi CRUD:** Seluruh *endpoint* utama yang meliputi pengambilan seluruh data, pengambilan data berdasarkan ID, penambahan data, pembaruan data, penghapusan data, serta filter berdasarkan **statusPengiriman** dapat berjalan sesuai dengan konvensi RESTful API.

3. **Konsistensi Respons JSON dan Kode Status:** API menghasilkan respons dalam format JSON. Operasi GET mengembalikan data secara langsung, sedangkan operasi POST, PUT, DELETE, dan penanganan *error* menggunakan struktur { status, message, data } dengan kode status HTTP yang sesuai, yaitu **200 OK, 201 Created, 400 Bad Request, dan 404 Not Found**.

4. **Validasi Data dan Error Handling:** Validasi berhasil digunakan untuk memastikan data paket kiriman yang dikirim memenuhi ketentuan, seperti kelengkapan nomorResi, pengirim, penerima, beratKg, dan statusPengiriman. Selain itu, *middleware catch-all* 404 memastikan *endpoint* yang tidak tersedia

tetap memberikan respons dalam format JSON.

5. **Deployment dan Version Control:** Proyek RESTful API **Ekspedisi – Paket Kiriman** telah disiapkan untuk pengelolaan menggunakan Git/GitHub dan deployment menggunakan Vercel. Pengujian terhadap berbagai skenario *endpoint* menunjukkan bahwa fungsi CRUD, filter, validasi, serta penanganan kesalahan dapat berjalan sesuai dengan kebutuhan tugas.

Lampiran

Tautan Repository GitHub:
https://github.com/gerz03/tugas1-restful-2428240067

Tautan Deployment:
https://tugas1-restful-2428240067.vercel.app/